SQL COMPLETE STUDY BOOK

MySQL Theory + Practical Code + Concepts

Beginner-friendly guide in simple Hinglish

Topics covered

- SQL basics: database, table, data types, INSERT, SELECT
- Transactions: START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT
- JOINs: INNER, LEFT, RIGHT, exclusive joins, FULL OUTER alternative
- Subqueries: IN, NOT IN, MAX, AVG, EXISTS, NOT EXISTS, HAVING
- Views: CREATE, REPLACE, ALTER, CHECK OPTION, DROP
- Indexes: normal, unique, composite, SHOW INDEX, EXPLAIN
- Stored Procedures: IN, OUT, INOUT, IF/ELSE, WHILE, CALL

Read the theory, run the code, then solve the practice questions.

Is book ko kaise use karein

Bhai, ye book MySQL beginner practice ke liye hai. Har chapter mein concept, code aur important points hain. SQL keywords uppercase mein hain taaki code readable rahe.

Important: Pehli baar chapter ka code top-to-bottom run karo. Dobara run karne par CREATE TABLE ya CREATE PROCEDURE error de sakta hai agar object pehle se exist karta ho. INSERT dobara run karne se duplicate rows aa sakti hain.

Contents

Chapter	Topic
1	SQL basics and first database
2	Transactions and savepoints
3	JOINs
4	Subqueries
5	Views
6	Indexes
7	Stored procedures
8	Quick revision

Basic SQL commands

Command	Kaam
CREATE DATABASE	Database banata hai
USE	Database select karta hai
CREATE TABLE	Table/columns banata hai
INSERT	Rows add karta hai
SELECT	Data read karta hai
UPDATE	Existing data change karta hai
DELETE	Rows remove karta hai
DROP	Database object remove karta hai

Chapter 1 — SQL Basics

SQL ka full form Structured Query Language hai. Iska use relational database mein data create, read, update aur delete karne ke liye hota hai.

Data types aur keys

• INT: whole number, jaise 1 ya 25.

• VARCHAR(50): text, maximum 50 characters.

• DECIMAL(10,2): exact decimal value, jaise 500.00; money values ke liye useful.

• PRIMARY KEY: har row ko uniquely identify karti hai.

• AUTO_INCREMENT: MySQL ID automatically generate karta hai.

Complete starter example

```
CREATE DATABASE IF NOT EXISTS sql_basics;
USE sql_basics;

CREATE TABLE accounts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50),
    balance DECIMAL(10,2)
);

INSERT INTO accounts (name, balance) VALUES
('Adam', 500.00),
('Bob', 300.00),
('Charlie', 1000.00);

SELECT * FROM accounts;

UPDATE accounts
SET balance = balance + 100
WHERE id = 1;

-- DELETE FROM accounts WHERE id = 3;
```

INSERT mistakes: table name exactly match karo; last column ke baad extra comma nahi; multiple rows ke beech commas; CREATE TABLE ke liye () use karo, {} nahi.

Safety tip: UPDATE/DELETE se pehle SELECT ... WHERE ... run karke confirm karo ki correct row target ho rahi hai.

Chapter 2 — Transactions

Transaction SQL statements ka group hai jise ek logical unit ki tarah handle kiya jata hai. Banking example mein debit aur credit ko ek transaction mein rakh sakte hain.

Command	Meaning
START TRANSACTION	Transaction start
COMMIT	Changes save
ROLLBACK	Uncommitted changes cancel
SAVEPOINT name	Checkpoint
ROLLBACK TO SAVEPOINT name	Checkpoint ke baad ke changes undo

Debit-credit example

```
CREATE DATABASE IF NOT EXISTS transaction_practice;
USE transaction_practice;

CREATE TABLE accounts (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    balance DECIMAL(10,2)
);

INSERT INTO accounts VALUES
(1, 'Adam', 500.00),
(2, 'Bob', 300.00),
(3, 'Charlie', 1000.00);

START TRANSACTION;

UPDATE accounts SET balance = balance - 50
WHERE id = 1 AND balance >= 50;

UPDATE accounts SET balance = balance + 50
WHERE id = 2;

COMMIT;
SELECT * FROM accounts;
```

Expected if run once from fresh data: Adam 450.00, Bob 350.00, Charlie 1000.00.

SAVEPOINT example

```
START TRANSACTION;

UPDATE accounts SET balance = balance + 1000 WHERE id = 1;
SAVEPOINT after_wallet_topup;

UPDATE accounts SET balance = balance + 50 WHERE id = 2;

ROLLBACK TO SAVEPOINT after_wallet_topup;
COMMIT;

SELECT * FROM accounts;
```

Adam ka +1000 update save hota hai. Bob ka +50 savepoint ke baad hua, isliye rollback se undo hota hai.

Important: Transaction support ke liye InnoDB jaise transactional engine ka use hona chahiye. Real money transfer mein dono rows aur affected rows verify karke hi COMMIT karein.

Chapter 3 — SQL JOINs

JOIN related columns ke basis par do tables ka data combine karta hai.

```
CREATE DATABASE IF NOT EXISTS join_practice;
USE join_practice;

CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    department_id INT
);

CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50)
);

INSERT INTO employees VALUES
(1,'Adam',101),(2,'Bob',102),(3,'Charlie',103),
(4,'David',104),(5,'Eve',105);

INSERT INTO departments VALUES
(101,'IT'),(102,'HR'),(103,'Finance'),(106,'Marketing');
```

INNER JOIN — only matching

```
SELECT e.id, e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Result: Adam-IT, Bob-HR, Charlie-Finance. Unmatched records nahi aate.

LEFT JOIN — all left rows

```
SELECT e.id, e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Employees left table hai, isliye David aur Eve bhi aayenge; unka department_name NULL hoga.

RIGHT JOIN — all right rows

```
SELECT e.name, d.department_id, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

Marketing bhi aayega, aur uske employee columns NULL honge.

LEFT EXCLUSIVE — left ke unmatched rows

```
SELECT e.id, e.name, e.department_id
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Result: David (104), Eve (105). MySQL mein LEFT EXCLUSIVE JOIN separate keyword nahi; LEFT JOIN + IS NULL pattern hai.

RIGHT EXCLUSIVE — right ke unmatched rows

```
SELECT d.department_id, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id
WHERE e.id IS NULL;
```

Result: Marketing (106).

FULL OUTER JOIN — MySQL alternative

```
SELECT e.id, e.name, d.department_id, d.department_name
FROM employees e LEFT JOIN departments d
ON e.department_id = d.department_id
UNION
```

```
SELECT e.id, e.name, d.department_id, d.department_name
FROM employees e RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

MySQL FULL OUTER JOIN directly support nahi karta. LEFT + RIGHT JOIN ko UNION se combine kar sakte hain.

JOIN	Meaning
INNER	Only matching rows
LEFT	All left rows + matches
RIGHT	All right rows + matches
LEFT exclusive	Left rows with no right match
RIGHT exclusive	Right rows with no left match
FULL alternative	Both sides, matching and unmatched

Chapter 4 — Subqueries

Subquery ek query ke andar doosri query hoti hai. Inner query ka result outer query use karti hai.

```
CREATE DATABASE IF NOT EXISTS subquery_practice;
USE subquery_practice;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50)
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

INSERT INTO customers VALUES
(1,'Adam','Delhi'),(2,'Bob','Mumbai'),(3,'Charlie','Pune'),
(4,'David','Delhi'),(5,'Eve','Mumbai');

INSERT INTO orders VALUES
(101,1,5000),(102,2,3000),(103,1,2000),
(104,3,7000),(105,4,1000),(106,2,4000);
```

IN — customers with orders

```
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
```

NOT IN — customers without orders

```
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

Is sample mein Eve ne order nahi kiya. General rule: NOT IN subquery mein NULL aa sakta ho to unexpected result ho sakta hai; NOT EXISTS consider karo.

MAX and AVG

```
SELECT * FROM orders
WHERE amount = (SELECT MAX(amount) FROM orders);

SELECT * FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

GROUP BY and HAVING

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 5000;
```

GROUP BY rows ko group karta hai; SUM total nikalta hai; HAVING aggregate ke baad groups filter karta hai.

Subquery in FROM

```
SELECT c.customer_name, totals.total_amount
FROM customers c
JOIN (
    SELECT customer_id, SUM(amount) AS total_amount
    FROM orders
    GROUP BY customer_id
) totals ON c.customer_id = totals.customer_id;
```

EXISTS and NOT EXISTS

```
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

SELECT * FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Keyword	Use
IN	Value list mein hai
NOT IN	Value list mein nahi; NULL se caution
MAX()	Maximum
AVG()	Average
SUM()	Total
EXISTS	Matching row exists
NOT EXISTS	Matching row does not exist
HAVING	Groups/aggregate results filter

Chapter 5 — Views

VIEW saved SELECT query ki tarah hota hai jo virtual table ke roop mein dikhta hai. Aam view separate copy of data nahi rakhta; underlying tables se data dikhata hai.

```
CREATE DATABASE IF NOT EXISTS views_practice;
USE views_practice;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(50),
    city VARCHAR(50),
    age INT
);
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    product_name VARCHAR(50),
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

INSERT INTO customers (customer_name, city, age) VALUES
('Adam','Delhi',25),('Bob','Mumbai',30),('Charlie','Pune',22),
('David','Delhi',35),('Eve','Mumbai',28);

INSERT INTO orders (customer_id, product_name, amount) VALUES
(1,'Laptop',50000),(2,'Mobile',20000),(1,'Keyboard',1500),
(3,'Monitor',12000),(4,'Mouse',800),(2,'Headphones',2500);
```

Simple view and WHERE view

```
CREATE VIEW customer_view AS
SELECT customer_id, customer_name, city FROM customers;
SELECT * FROM customer_view;

CREATE VIEW delhi_customers AS
SELECT customer_id, customer_name, city, age
FROM customers WHERE city = 'Delhi';
SELECT * FROM delhi_customers;
```

JOIN view and aggregate view

```
CREATE VIEW customer_orders AS
SELECT c.customer_id, c.customer_name, o.order_id, o.product_name, o.amount
FROM customers c JOIN orders o ON c.customer_id = o.customer_id;

CREATE VIEW customer_total_spending AS
SELECT c.customer_id, c.customer_name, SUM(o.amount) AS total_spending
FROM customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name;

SELECT * FROM customer_orders;
SELECT * FROM customer_total_spending;
```

Replace, alter, check option

```
CREATE OR REPLACE VIEW customer_view AS
SELECT customer_id, customer_name, city, age FROM customers;

ALTER VIEW customer_view AS
SELECT customer_id, customer_name, city FROM customers;

CREATE OR REPLACE VIEW young_customers AS
SELECT customer_id, customer_name, city, age
FROM customers WHERE age < 30
WITH CHECK OPTION;
```

WITH CHECK OPTION updatable view ke through changes ko view ki WHERE condition satisfy karne par majboor karta hai. JOIN, GROUP BY aur aggregate views par INSERT/UPDATE/DELETE restrictions ho sakti hain.

Inspect and drop

```
SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
SHOW CREATE VIEW customer_view;
DROP VIEW IF EXISTS delhi_customers;
```

Command	Use
CREATE VIEW	View create
CREATE OR REPLACE VIEW	Create or replace
ALTER VIEW	Definition change
WITH CHECK OPTION	View condition enforce
SHOW CREATE VIEW	Definition show
DROP VIEW	Remove view

Chapter 6 — Indexes

Index data search karne mein help karta hai. Search faster ho sakti hai, lekin index storage leta hai aur INSERT/UPDATE/DELETE ko slower bana sakta hai.

```
CREATE DATABASE IF NOT EXISTS index_practice;
USE index_practice;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(50),
    email VARCHAR(100),
    city VARCHAR(50),
    age INT
);

INSERT INTO customers (customer_name, email, city, age) VALUES
('Adam','adam@example.com','Delhi',25),
('Bob','bob@example.com','Mumbai',30),
('Charlie','charlie@example.com','Pune',22),
('David','david@example.com','Delhi',35),
('Eve','eve@example.com','Mumbai',28),
('Frank','frank@example.com','Bhopal',26);
```

Simple, unique and composite index

```
CREATE INDEX idx_customer_name ON customers(customer_name);
CREATE INDEX idx_customer_city ON customers(city);
CREATE UNIQUE INDEX idx_customer_email ON customers(email);
CREATE INDEX idx_city_age ON customers(city, age);

SELECT * FROM customers WHERE customer_name = 'Adam';
SELECT * FROM customers WHERE city = 'Delhi';
SELECT * FROM customers WHERE city = 'Delhi' AND age = 35;

SHOW INDEX FROM customers;

EXPLAIN SELECT * FROM customers WHERE customer_name = 'Adam';
```

PRIMARY KEY ka index automatically banta hai. UNIQUE INDEX duplicate non-NULL email ko rokta hai. Composite index ka column order matter karta hai: (city, age) city-based search aur city+age filtering mein useful ho sakta hai.

Drop indexes

```
DROP INDEX idx_customer_name ON customers;
DROP INDEX idx_customer_city ON customers;
DROP INDEX idx_city_age ON customers;
DROP INDEX idx_customer_email ON customers;
SHOW INDEX FROM customers;
```

Har column par index mat banao. Real query patterns aur EXPLAIN ke basis par decide karo.

Chapter 7 — Stored Procedures

Stored Procedure SQL statements ka saved group hai jise CALL se execute kar sakte hain. DELIMITER // MySQL client mein procedure definition ke semicolons ko handle karne mein help karta hai.

Note: Kuch SQL editors DELIMITER command accept nahi karte. MySQL Workbench mein stored routine editor bhi use kar sakte ho.

```
CREATE DATABASE IF NOT EXISTS procedure_practice;
USE procedure_practice;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(50),
    city VARCHAR(50),
    balance DECIMAL(10,2)
);

INSERT INTO customers (customer_name, city, balance) VALUES
('Adam','Delhi',5000),('Bob','Mumbai',3000),
('Charlie','Pune',7000),('David','Delhi',2000),('Eve','Mumbai',9000);
```

Simple procedure

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM customers;
END //
DELIMITER ;

CALL GetAllCustomers();
```

IN parameter

```
DELIMITER //
CREATE PROCEDURE GetCustomerById(IN p_id INT)
BEGIN
    SELECT * FROM customers WHERE customer_id = p_id;
END //
DELIMITER ;

CALL GetCustomerById(1);
```

OUT parameter

```
DELIMITER //
CREATE PROCEDURE GetCustomerCount(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM customers;
END //
DELIMITER ;

CALL GetCustomerCount(@total);
SELECT @total AS total_customers;
```

INOUT parameter

```
DELIMITER //
CREATE PROCEDURE AddBonus(INOUT amount DECIMAL(10,2))
BEGIN
    SET amount = amount + 500;
END //
DELIMITER ;

SET @amount = 1000;
CALL AddBonus(@amount);
SELECT @amount AS updated_amount;
```

IF / ELSE

```
DELIMITER //
CREATE PROCEDURE CheckBalance(IN p_id INT)
BEGIN
    DECLARE customer_balance DECIMAL(10,2);
    SELECT balance INTO customer_balance
    FROM customers WHERE customer_id = p_id;

    IF customer_balance >= 5000 THEN
```

```
        SELECT 'High Balance' AS message;
    ELSE
        SELECT 'Low Balance' AS message;
    END IF;
END //
DELIMITER ;

CALL CheckBalance(1);
```

UPDATE procedure

```
DELIMITER //
CREATE PROCEDURE AddMoney(IN p_id INT, IN p_amount DECIMAL(10,2))
BEGIN
    UPDATE customers
    SET balance = balance + p_amount
    WHERE customer_id = p_id;
END //
DELIMITER ;

CALL AddMoney(1, 1000);
```

WHILE loop

```
DELIMITER //
CREATE PROCEDURE PrintNumbers()
BEGIN
    DECLARE i INT DEFAULT 1;
    WHILE i <= 5 DO
        SELECT i AS number;
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;

CALL PrintNumbers();
```

Aggregate OUT, inspect and drop

```
DELIMITER //
CREATE PROCEDURE GetTotalBalance(OUT total_balance DECIMAL(12,2))
BEGIN
    SELECT SUM(balance) INTO total_balance FROM customers;
END //
DELIMITER ;

CALL GetTotalBalance(@total_balance);
SELECT @total_balance AS total_balance;

SHOW PROCEDURE STATUS WHERE Db = 'procedure_practice';
SHOW CREATE PROCEDURE GetAllCustomers;
DROP PROCEDURE IF EXISTS GetAllCustomers;
```

Term	Meaning
IN	Input parameter
OUT	Output parameter
INOUT	Input and output
CALL	Execute procedure
DECLARE	Local variable
IF/ELSE	Condition
WHILE	Loop

Chapter 8 — Quick Revision

Concept	Remember
Database	Organized container for data
Table	Rows and columns
Primary key	Unique row identity
Transaction	START TRANSACTION then COMMIT/ROLLBACK
SAVEPOINT	Checkpoint inside transaction
INNER JOIN	Only matching rows
LEFT JOIN	All left rows
LEFT exclusive	LEFT JOIN + WHERE right_key IS NULL
Subquery	Query inside another query
EXISTS	Check matching row existence
VIEW	Saved SELECT / virtual table
INDEX	Search helper with storage/write cost
Stored procedure	Saved SQL logic called using CALL

Common errors and fixes

• Unknown column: spelling check karo, jaise balance (not balace).

• Table not found: USE correct database; SHOW TABLES run karo.

• Table already exists: code dobara run ho sakta hai; object carefully reset karo.

• Duplicate rows: INSERT repeat hui hai; INSERT dobara run mat karo.

• Balance unchanged: UPDATE query ka error, WHERE ID, aur affected rows check karo.

• JOIN mein NULL: right-side match nahi mila.

• NOT IN unexpected: subquery mein NULL ho sakta hai; NOT EXISTS try karo.

• Procedure syntax error: DELIMITER support aur BEGIN/END structure check karo.

Practice questions

• Accounts table banao aur 3 rows insert karo.

• Adam se 100 debit aur Bob ko 100 credit karne ki transaction likho.

• SAVEPOINT ke baad update karo aur ROLLBACK TO SAVEPOINT se undo karo.

• Employees/departments ke liye INNER JOIN aur LEFT JOIN likho.

• Aise customers nikalo jinhone order nahi kiya.

• Average order amount se zyada orders nikalo.

• Delhi customers ke liye view banao.

• Email par UNIQUE INDEX banao aur SHOW INDEX run karo.

• IN parameter ke saath customer ID search procedure banao.

Suggested order: SELECT/WHERE → INSERT/UPDATE/DELETE → JOINs → Subqueries → Transactions → Views/Indexes → Stored Procedures.

End of book — daily practice karo aur har query ka result khud verify karo.